Very brief intro to R programming

These are the most important aspects of R programming you need to know:

- Comments are preceeded by

  ```
  #
  ```

- Assigning a value to a variable using the

  ```
  <-
  ```

  operator (and not "=" as in most other programming languages).
  E.g.

  ```
  a<-1.0
  ```

  will cause the variable "a" to contain "1.0"

- Use

  ```
  print( argument )
  ```

  to print the value of the argument in the R-shell when running a script.

- There are many ways of constructing arrays in R. E.g.

  ```
  x <- seq(0,3,0.5)
  y <- vector(mode="numeric",length=10)
  z <- c(1,2,3)
  m <- matrix(0.0,nrow=3,ncol=4)
  ```

- Individual elements and sub-arrays are extracted using the "[]" and ":" operators. E.g. "x[1]" is the first element of x which contains 0.0 and "x[1:2]" is another array which contains (0.0,0.5).

- Write to sub-arrays:

```
m[1,2] <- 1.2
m[2,1:2] <- c(3.2,3.3)
```

- Data frames: Data frames are useful for organizing and analyzing tabular data with columns of potentially different data types.

```
x <- c(1,2,3,4)
y <- c("red","white","red,NA)
z <- c(TRUE,TRUE,TRUE,FALSE)
mydata <- data.frame(x,y,z)
names(mydata) <- c("ID","Color",
"Passed")
```

- Basic flow control: Conditional execution

  ```
  if(){ } else { }
  ```

- Notice "==" logical equal, "!=" logical not equal.

- Basic flow control: For loops

  ```
  for(i in vector){
  statements
  if(should stop) break
  }
  ```

- Important: If possible avoid using for-loop by using vectorized operations.

  ```
  y <- x + 2.0
  ```

  adds 2.0 to each element in vector x and stores the result in vector y.

- Function syntax

```
functionName <- function(arguments) {
statements
...
return(value) #not obligatory
}
```

- Lists: Lists are useful as collections of different data types. Use "$" to access the fields

```
l <- list(a=c(1.2,2.2),b=TRUE,c=1)
l$a $ prints contents of first field
l$c $ prints contents of third field
l[[2]] $ prints TRUE
```

  In particular useful for returning multiple things from functions

```
f <- function(a){
asq <- a^2
return(list(a=a,aTimes2=2.0*a,
aSquared=asq))
```

```
}
ret <- f(4.0)
ret$a # prints 4.0
ret$aSquared # prints 16.0
```

- (somewhat advanced:) You may use variables in encompassing scopes in functions, but cannot alter them, e.g.

```
a <- 1.0
print_a <- function(){print(a)}
increment_a <- function(b){
a <- a+b # "left hand side a"
# is only in function scope
return(a)
}
print_a() # prints 1.0
increment_a(3.0) # prints 4.0
print_a() # prints 1.0
a # prints 1.0
```